

NutriParse Reader – Documentation

Author: Amilton Domingos Alexandre Koxi

GitHub: <https://github.com/amiltonkoxi/nutriparsers-reader>

Live Demo: <https://nutriparsereader.vercel.app>

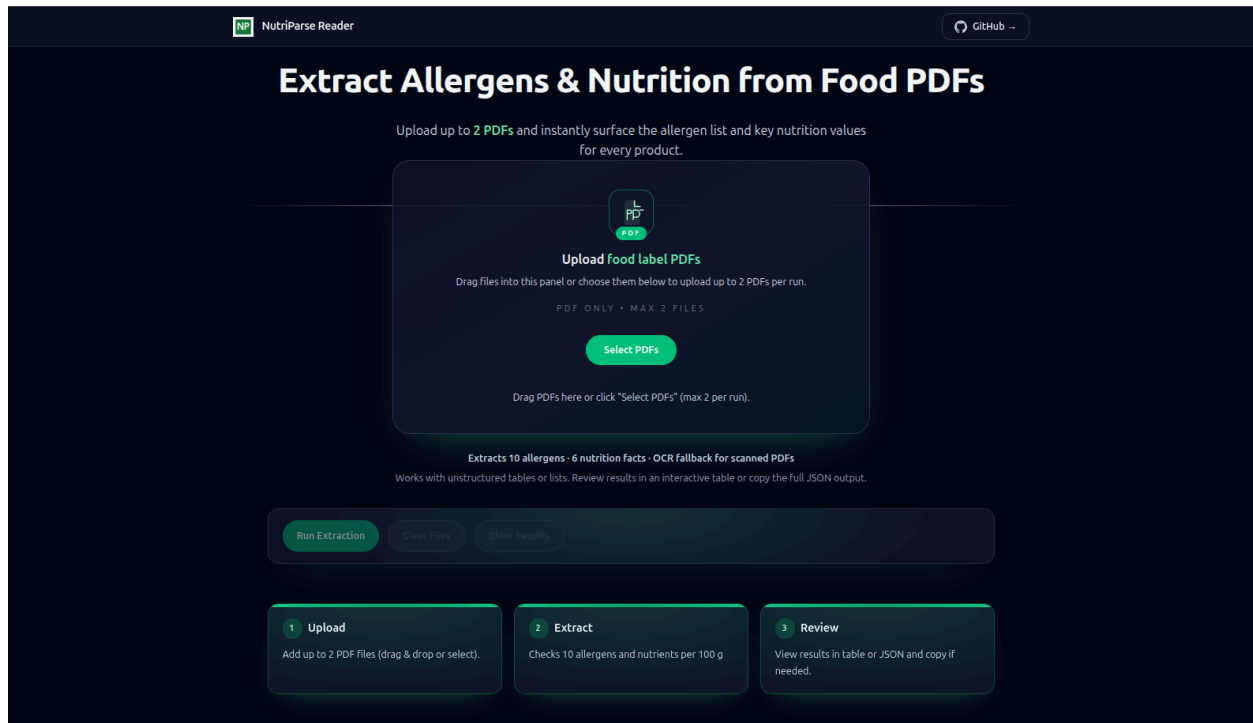


Image 1- Home page

1. Introduction

NutriParse Reader automates the extraction of **allergen statements** and **nutrition facts** from PDF product specifications. It supports both **text-based** and **scanned** PDFs using OCR and rule-based/AI-assisted parsing. The output is shown as interactive cards and as a **downloadable JSON**.

Scope: EU allergens (10) and core nutrition metrics (energy, fat, carbs, sugars, protein, salt).

Target users: The application is intended for quality assurance teams validating food labeling, nutrition analysts reviewing product specifications, retailers verifying product information before sales, and for academic demonstrations in research or classroom settings.

1.1 Quickstart

```
# clone
git clone https://github.com/amiltonkoxi/nutriparse-reader.git
cd nutriparse-reader

# 1) Backend (FastAPI)
cd backend
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
# Ubuntu OCR deps (if local, not Docker)
sudo apt update && sudo apt install -y tesseract-ocr poppler-utils
# run
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000

# 2) Frontend (Next.js)
cd ../frontend
npm install
printf "NEXT_PUBLIC_API_BASE=http://127.0.0.1:8000\n" > .env.local
npm run dev

Open: http://localhost:3000
```

2. System Architecture

2.1 High-Level Overview

Component	Technology	Hosting / Environment	Role
Frontend	Next.js / React / Tailwind	Vercel	Upload PDFs, visualize results, copy JSON
Backend	FastAPI / Python (Docker)	Render	REST API, OCR, parsing, confidence scoring

Data Flow: Browser → Vercel (frontend) → POST /extract → Render (backend) → OCR/Parsers → JSON → Frontend.

2.2 Frontend

- **Framework:** Next.js + React + Tailwind CSS
- **Config:** NEXT_PUBLIC_API_BASE → backend URL (local or Render)
- **Responsibilities:** dropzone, run extraction, tabs (Summary / JSON / Preview), copy JSON, queue ≤ 2 PDFs

2.3 Backend

- **Framework:** FastAPI (Python 3.12)
- **Core paths:**
 - backend/app/main.py – API, CORS, confidence
 - backend/extractors/ – OCR (Tesseract), text extraction (PyPDFium/pdfplumber)
 - backend/parsers/ – allergens & nutrition (regex + optional LLM)
 - backend/rules/ – regex/JSON heuristics
- **Endpoint:** POST /extract (multipart PDF → structured JSON)
- **Health:** GET /health

3. Installation & Local Setup

3.1 Prerequisites

- Node.js ≥ 18 (npm or yarn)
- Python ≥ 3.12
- Tesseract OCR + Poppler (if running locally without Docker)
- Docker (optional)

3.2 Backend (virtualenv)

```
cd backend
python3 -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt

# Install OCR deps (Ubuntu/Debian)
sudo apt update && sudo apt install -y tesseract-ocr poppler-utils

# Run dev
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Test health:

```
curl -fsS http://127.0.0.1:8000/health && echo "OK"
```

Test extract:

```
curl -sS -F "file=@/path/to/sample.pdf;type=application/pdf" \
  http://127.0.0.1:8000/extract | jq .
```

3.3 Frontend

```
cd frontend
npm install
printf "NEXT_PUBLIC_API_BASE=http://127.0.0.1:8000\n" > .env.local
npm run dev
```

Open: <http://localhost:3000>

4. Docker & Compose (optional local parity)

4.1 Backend: Docker build/run

```
# from backend/
docker build -t nutriparse-backend:local .
# expose port 8000
docker run --rm -p 8000:8000 \
  -e ALLOWED_ORIGINS="http://localhost:3000" \
  -e ALLOWED_ORIGIN_REGEX="^https://.*vercel\\.app$" \
  nutriparse-backend:local
```

4.2 Minimal docker-compose.yml (example)

```
version: "3.9"
services:
  api:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
      ALLOWED_ORIGINS: "http://localhost:3000"
      ALLOWED_ORIGIN_REGEX: "^https://.*vercel\\.app$"
```

Run:

```
docker compose up --build
```

5. Deployment

5.1 Frontend (Vercel)

1) Push your repo to GitHub

2) On Vercel: Import Project → link GitHub → set env

VERCEL_ENV VARS:

`NEXT_PUBLIC_API_BASE=https://<render-service>.onrender.com`

3) Deploy main branch

5.2 Backend (Render, Docker)

Service type: Web Service (Docker)

Context/Root: backend/

Healthcheck: /health

Environment variables:

`ALLOWED_ORIGINS=https://nutriparsereader.vercel.app,http://localhost:3000`

`ALLOWED_ORIGIN_REGEX=https://.*vercel\.app$`

Free tier note: cold starts ~30–60s.

5.3 CORS Middleware (reference)

from fastapi.middleware.cors **import** CORSMiddleware

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=ALLOWED_ORIGINS,
    allow_origin_regex=ALLOWED_ORIGIN_REGEX,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

6. API Usage

6.1 POST /extract

Request (multipart/form-data):

```
curl -sS -X POST \
  -F "file=@/path/to/sample.pdf;type=application/pdf" \
  https://<render-service>.onrender.com/extract | jq .
```

Response (schema excerpt):

```
{
  "meta": {
    "product_name": "...",
    "source_file": "sample.pdf",
    "extraction_mode": "text|ocr",
    "serving_basis": "per 100g|per serving",
    "languages": ["en", "hu"],
    "confidence": 0.0
  },
  "allergens": {
    "gluten": {"status": "contains|traces|absent|unknown", "evidence": "..."}
  },
  "nutrition": {
    "energy_kcal": 100,
    "fat_g": 10.0,
    "carbs_g": 0.0,
    "sugars_g": 0.0,
    "protein_g": 0.0,
    "salt_g": 0.0
  },
  "diagnostics": {"notes": ["..."], "raw_preview": "..."}
}
```

6.2 Health

curl -fsS https://<render-service>.onrender.com/health && echo OK

7. Configuration & Environment

Backend

```
export ALLOWED_ORIGINS="https://nutriparsereader.vercel.app,http://localhost:3000"
```

```
export ALLOWED_ORIGIN_REGEX="^https://.*vercel\\.app$"
```

Frontend

```
printf "NEXT_PUBLIC_API_BASE=https://<render-service>.onrender.com\n" >
```

```
frontend/.env.local
```

Languages supported: Hungarian, English (baseline), partial Polish/German.

Limits: up to 2 PDFs per run; large PDFs (>6 pages / >10MB) might timeout on free tiers.

8. Developer Guide

8.1 Project structure (excerpt)

nutriparse-reader/

```
├─ backend/
|   ├─ app/          # API FastAPI
|   ├─ extractors/   # OCR & PDF text
|   ├─ parsers/      # Allergen & nutrition
|   ├─ rules/        # Regex & heuristics
|   └─ requirements.txt
|
├─ frontend/
|   ├─ app/          # Next.js routing
|   ├─ components/   # UI components
|   ├─ lib/          # API client
|   └─ public/       # Static assets
|
└─ README.md
```

8.2 Extend parsing

- **Allergens:** edit rules/allergens.json, update logic in parsers/allergen_parser.py.
- **Nutrition:** adjust regex in parsers/nutrition_parser.py (unit normalization, per-100g mapping).
- **Confidence:** scoring rules in app/main.py.

8.3 Local helpers

health probe loop

```
for i in {1..20}; do \
  sleep 0.5; \
  curl -fsS http://127.0.0.1:8000/health >/dev/null && echo "✅ Backend OK" && break; \
  || [ $i -eq 20 ] && echo "❌ Backend not healthy"; \
done
```

pretty print last logs (if you run uvicorn with nohup)

```
tail -n 120 ./tmp/backend.log | sed -n '1,120p'
```

9. Troubleshooting

9.1 CORS errors in browser

- **Symptom:** Access-Control-Allow-Origin missing.
- **Fix:** Ensure ALLOWED_ORIGINS includes your Vercel URL and ALLOWED_ORIGIN_REGEX matches preview domains.

9.2 OCR returns empty / poor text

- **Symptom:** blank or garbled raw_preview for scans.
- **Fix:** confirm Tesseract installed; ensure scans $\geq$ **300 DPI**; try grayscale preprocessing; check language packs.

Install extra language packs (example HU)

`sudo apt install tesseract-ocr-hun`

9.3 Cold start latency (Render)

- **Symptom:** 30–60s first request.
- **Workaround:** warm-up pings, or move to paid tier.

9.4 Large PDFs time out

- **Symptom:** >6 pages or >10MB fail on free tier.
- **Fix:** split PDF before upload or raise timeouts.

split a big PDF into chunks of 3 pages

`pdftk big.pdf cat 1-3 output part1.pdf`

`pdftk big.pdf cat 4-6 output part2.pdf`

10. License & Credits

- **OCR:** Tesseract
- **PDF:** PyPDFium / pdfplumber
- **Web:** FastAPI, Next.js, Tailwind

11. Acknowledgments

University of Debrecen – Faculty of Informatics.

Special thanks to supervisors and peers for valuable feedback on allergen and nutrition parsing heuristics.